



P17

Least-Privilege Orchestration: Tightening and Re-granting the Transfer Bundle Scope

Least-Privilege Orchestration: Scope Tightening and
Re-authorization in the Transfer Bundle

TwinsEarth · 智能体宇宙论文系列

Least-Privilege Orchestration: Tightening and Re-granting the Transfer Bundle Scope

Abstract — A structural upgrade to P10, this paper adds a scope field to the Transfer Bundle so an owner can touch only the stages it is authorized for, following least privilege. It defines how scope is narrowed on handoff and explicitly re-granted when a task crosses a boundary. Cross-referenced with BFT-lite (P2), fault governance (P3), and the task market, the mechanism limits blast radius when an agent is compromised or mistaken and makes privilege changes auditable.

Keywords: least privilege; scope; transfer bundle; authorization; orchestration; blast radius

孪生地球

最小权限编排：TransferBundle 的 scope 收紧与重授

Working Title (EN): Least-Privilege Orchestration: Adding a Scope Field to the Transfer Bundle so Owners Can Only Touch the Stages They Are Authorized For

分片 D · 市场与架构篇 P17 · v7.5.0 P10 《Agent-as-Tool 与 TransferBundle》的结构升级 · 与 P2 《BFT-lite 停止决策》、P3 《故障治理》、P11 《内部任务市场》互引 · UDOS 推演引擎 v7.6.0 · 房文新 · 2026-09-21

一句话先说清：多智能体编排真正的管理瓶颈，不是 Leader“看不懂”子 Agent 在干什么，而是 Leader 把**过大的权限**一把交给了子 Agent；v7.5.0 的 TransferBundle 只有一个 owner（谁负责），v7.6.0 给它加上一个 scope（这个 owner **被允许操作哪些 stage**），让权限随推进自动收紧、随交接重新授予——这是“最小权限”这条老工程原则第一次被写进 UDOS 的交接契约里。

证据口径（全文只声明这一次）：本文是**设计提案**，不含新的 UDOS 实测数字；scope 字段的收益一律标“设计提案·待实施”，用 [RESULT NEEDED] 占位，不写成实测。外部唯一对话对象是 ClawArena-Team (arXiv: 2606.31174, 存在性已联网核验于 2026-09-21)；其“管理瓶颈在权限授予”“成本与管理质量近似解耦”“纯执行式评分、无 LLM 评判”为论文方向，而“无 LLM 超过 50% 工作区权限精度”“成本跨度逾百倍而总分跨度不足四倍”“SMS=任务正确性×最小权限因子×模态路由因子”等**精确阈值与公式字符串未在摘要级核验中逐字命中**，本文一律按“论文报告·未独立复核”转述，不写死具体百分比。UDOS 旧数字只回指 v7.5.0 各篇（P10 六字段契约、26 项测试 verified）。

摘要

当一个 Leader Agent 同时管理若干子 Agent 时，工程上最容易被忽视的失败点不是“任务会不会做”，而是“它被允许做多少”。ClawArena-Team (arXiv:2606.31174) 在一组多轮、多模态、多目录的子 Agent 编排基准上报告：Leader 的管理瓶颈集中在**权限授予**环节，且 API 成本与管理质量近似解耦——花更多钱换更强的 Leader 模型，并不能等比例换得更好的编排质量。这两条经验指向同一个结构判断：**管理是一个独立于模型能力的维度，且它的第一抓手是权限边界，而不是感知或推理。**

v7.5.0 的 TransferBundle 把交接包固定为六字段 (Goal/Context/Done/Trace/Owner)，其中 owner 只回答“这一棒现在归谁负责”，不回答“它被允许动哪些东西”。本文提出 v7.6.0 的最小权限升级：在 owner 旁新增 scope 字段，显式记录“该 owner 当前被允许操作的 stage 集合”。advance() 把已完成的 stage 从 scope 中移除（**自动收紧**），handoff() 把控制权交给下一棒时由新 owner 重新申领并校验一个更小的 scope（**重授**）。任何越出 scope 的写操作在 validate() 阶段即被机械拒绝，返回类型化错误 scope_violation，而不是静默放行。本文给出该字段的契约定义、三个不变量（scope 单调收紧、越界即拒、重授必须经 validate）、与

P10 既有六字段/26 项测试的兼容路径，并把"最小权限是否真的降低越界与误隔离"留给一个预注册因子实验。本文诚实声明：scope 是**设计提案·待实施**，其降本/降错收益目前没有 UDOS 实测数据，凡涉具体数字处一律 [RESULT NEEDED]。

关键词：多智能体编排；最小权限；TransferBundle；权限收紧；交接契约；故障治理；管理瓶颈；设计提案

结构化摘要（背景问题 → 方法/论点 → 证据 → 贡献）

- **背景问题：**多智能体编排长期把"管理难"归因于"模型不够聪明"，于是默认解法是"换更强、更贵的 Leader"。但 ClawArena-Team 的执行式评分提示：管理瓶颈其实落在**权限授予**，且成本与管理质量近似解耦——换贵模型不是提升编排质量的有效杠杆。
- **方法/论点：**把"最小权限"（least privilege，安全工程里"只给完成当前任务所需的最小权限"的老原则）从口号写成 TransferBundle 的一个新字段 scope；权限随 advance 自动收紧、随 handoff 重授、越界即被 validate 机械拒绝。
- **证据线索：**(1) ClawArena-Team 的论文方向（权限瓶颈、成本-质量解耦、无 LLM 评判的执行式评分，论文报告·未独立复核）；(2) v7.5.0 P10 已 verified 的六字段契约与 26 项测试（scope 是其上的增量，不推翻既有不变量）；(3) 安全工程最小权限原则（教科书级共识，本文不声称是其首创）。
- **贡献：**(1) 把"owner=谁负责"升级为"owner+scope=谁负责、能碰哪些 stage"；(2) 给出三条可被测试钉死的契约不变量；(3) 提供与 v7.5.0 兼容的迁移路径与预注册因子实验设计，并诚实标注全部收益为 [RESULT NEEDED]。

1 引言：管理瓶颈不在"看不看得懂"，在"给了多少权限"

多智能体系统从"几个 Agent 协作"走向"一个 Leader 管一群子 Agent"时，一种普遍直觉是：管不好，是因为 Leader 不够强；解法就是把 Leader 换成更强、更贵的模型。这种直觉把"管理"当成了"推理能力"的副产品。

ClawArena-Team (arXiv:2606.31174) 在子 Agent 编排基准上给出了一个值得认真对待的反方向信号。它用**纯执行式评分**（即按任务实际产出打分，**不引入 LLM-as-Judge**，避免裁判模型自身的偏见）测量 Leader 的管理能力，并报告了两点（论文报告·未独立复核）：

1. 管理瓶颈集中在**权限授予**——Leader 不是"看不懂子 Agent 在做什么"，而是"给了子 Agent 过大的工作区权限"，越权操作因此高发；
2. API 成本与管理质量**近似解耦**——成本跨度很大，而管理总分跨度远小于成本跨度，即"贵模型=好管理"这一等式不成立。

这两点合起来指向一个结构判断：**管理是一个独立于模型能力的维度，它的第一个抓手是"权限边界"，而不是"感知广度"或"推理深度"**。如果这个判断成立，那么正确的工程动作不是"把 Leader 做大"，而是"把权限切小、切准、随流程收紧"。

本文做的就是这件事。v7.5.0 的 TransferBundle (P10) 已经把交接写成可机械校验的六字段契约，但其中 owner 字段只回答"这一棒归谁"，不回答"它被允许动哪些 stage"。本文在其上加一个 scope，把"最小权限"这条安全工程老原则第一次写进交接契约。

需要先划清三件事：第一，本文不重新证明最小权限原则（它是安全工程教科书共识），本文只回答"它在 UDOS 的交接契约里长什么样"；第二，本文**不声称** scope 已经被实测降本降错——它是设计提案；第三，本文不把 ClawArena 的精确阈值当成自己的实验数据，只借它的方向作为设计动机。

2 从 owner 到 owner+scope：一次字段级的结构升级

2.1 v7.5.0 的 owner 缺了什么

回顾 P10：一次交接由发送方构建 `TransferBundle`，六字段为 `goal`（目标）、`context`（事实字典）、`done`（已完成阶段列表）、`todo`（待办阶段列表）、`trace`（事件序列号列表）、`owner`（当前责任方）。

`handoff(next_owner)` 先调用 `validate()`，有问题就不转移责任，没问题才把 `owner` 改为 `next_owner`。

这套契约很扎实，但它隐含一个假设：只要 `owner` 是"对的人"，它就能对整个 `todo` 里的东西下手。`owner` 是一个单点责任指针，它保证了"有人负责"，却没有保证"这个人只能负责他该碰的那一格"。换句话说，P10 解决了"无人收口"，但没有解决"有权限越界"——一个被错误地推上 `owner` 位置的 Agent，或一个被注入错误指令的诚实 Agent，在 `owner` 模式下仍能对全部待办阶段执行写操作。

这正是 ClawArena 所诊断的失败面：责任边界 $\neq$ 权限边界。

2.2 新增的 scope 字段

本文提议把六字段升级为七字段，在 `owner` 旁新增：

- `scope`：当前 `owner` 被允许操作的 `stage` 集合，是 `todo` 的一个真子集（允许为空集，空集即"只读观察、不得写"）。

一个更直观的写法：v7.5.0 的 `owner` 回答"这一棒归谁"，v7.6.0 的 `owner+scope` 回答"这一棒归谁、以及它此刻被允许碰哪些格子"。

`scope` 不是静态配置，而是随工单推进动态变化的：

- `advance(bundle, stage, ...)` 成功后，把 `stage` 从 `scope` 中移除——任务做完了，就立刻收回对该 `stage` 的写权限。这叫**自动收紧**（`scope shrinks monotonically as stages advance`）。
- `handoff(next_owner)` 时，新 `owner` 不是"继承全部 `todo` 的权限"，而是**重新申领**一个它声明需要的 `new_scope`；只有当 `new_scope  $\subseteq$  todo` 且通过 `validate()` 时，控制权与新 `scope` 一并转移。这叫**重授**（`re-authorization on handoff`）。
- 任何越出当前 `scope` 的写操作（例如某 `owner` 试图操作一个不在它 `scope` 内的 `stage`），在 `validate()` 阶段即被拒绝，返回类型化错误 `scope_violation`，并把违规 `stage`、当前 `scope`、请求者记入 `trace`。它**不静默放行**，也不**静默失败**——和 P10 拒绝静默转发是同一个精神。

2.3 三条契约不变量

`scope` 的价值必须落在可测试的不变量上，而不是"感觉更安全"。本文提出三条：

- **I1（scope 单调收紧）**：在同一个 `owner` 任期内，`scope` 只能随 `advance` 缩小或不变，不能被悄悄放大。形式上，相邻两次 `advance` 后满足 `scope_{t+1}  $\subseteq$  scope_t`。
- **I2（越界即拒）**：任何对 `s  $\notin$  scope` 的写操作都返回 `scope_violation`，且不产生副作用、不改变 `done/todo/owner`。
- **I3（重授必过 validate）**：`handoff` 转移的新 `scope` 必须满足 `new_scope  $\subseteq$  todo` 且非空（除非显式的只读交接），否则不转移责任，与 P10"validate 失败不换 `owner`"完全同构。

这三条和 P10 既有的"handoff 失败不换 `owner`"、"`done  $\cap$  todo` 被拒"、"缺 `owner  $\rightarrow$  no_closer`"是并列关系，不是替代关系。它们都应被新增的单元测试钉死，证据等级目标为 `verified`（与 P10 的 26 项测试同标准）。

3 为什么是"权限"而不是"模型大小"

图1 权限边界与责任边界的分离

图1 左：v7.5.0 的 owner 是"单点责任指针"，谁负责=谁可对全部待办下手；右：v7.6.0 的 owner+scope 把责任与权限解耦——谁负责、以及它此刻被允许碰哪些格子，分开记录。概念示意，非实测数据。

3.1 成本与质量解耦意味着什么

ClawArena-Team 的执行式评分报告（论文报告·未独立复核）给出一个对工程排期很重要的提示：在它的场景里，Leader 模型的成本跨度很大，但管理质量的总分跨度远小于成本跨度。这不是说"模型大小完全没用"，而是说**"换更大的 Leader 模型"不是提升编排质量的性价比杠杆**——如果你真正的瓶颈是权限越界，那么把 Leader 从便宜模型换成旗舰模型，它依然会把子 Agent 的工作区开得过大，因为问题不在它"会不会推理"，而在系统"给了它多大的手"。

用 P0b 的话说，这又是一次**"结构先验决定上限"**的印证：owner 字段这种结构选择，先于模型大小决定了系统会在哪一类失败上翻车。

3.2 执行式评分：不把裁判权交给另一个 LLM

ClawArena 另一个值得借鉴的工程选择是**纯执行式评分、无 LLM 评判**。多智能体评估里，"让一个 LLM 当裁判给别的 LLM 打分"（LLM-as-Judge）很常见，但裁判模型本身有偏好、有立场，会把新的噪声引进评估。执行式评分按任务**实际产出**判分，把裁判从"模型自评"换成"产出是否达成"。

这与 UDOS 一贯坚持的"用机械契约与可复跑测试代替模型自评"是一致的（P10 的 `validate()`、P12 的证据分级都是这个取向）。本文对 scope 的验收同样不走"让 Leader 自证管理得当"，而是走"scope_violation 是否被机械触发、越界是否真的被拒"这类可被测试断言钉死的判据。

4 scope 的动态生命周期：收紧与重授

图2 scope 随 advance 收紧、随 handoff 重授

图2 一次工单推进的 scope 轨迹：交接时新 owner 申领一个小 scope（重授），每完成一个 stage 就把它从 scope 移除（自动收紧），越界写操作在 validate 被拒并记 trace。概念示意，非实测数据。

把一次工单从下发到收口的 scope 轨迹画出来（图2），可以看到它在两个动作之间往复：

1. **重授 (handoff)**：控制权交给下一棒时，新 owner 声明 `new_scope`。这里有一个关键设计选择——为什么不让它自动继承全部 `todo`？因为"自动继承"等于把权限决策交还给了流程默认值，又回到了 owner 模式的老路。强制"申领"的意义在于：**让"这个 owner 需要碰哪些 stage"成为一个被显式声明、可被审计、可被拒绝的动作**，而不是一个隐式默认。
2. **自动收紧 (advance)**：stage 做完即收回。这一步几乎零成本，因为 `advance` 本来就要改 `todo` → `done`，把它在 `scope` 里同步移除只是同一处状态更新的附带动作。它的价值是**让权限随事实变化**：一个 owner 不会因为"曾经负责过某 stage"而永久保留对它的写权。
3. **越界拒绝 (validate)**：这是 scope 真正起防御作用的地方。当一个被误导或被注入指令的 owner 试图操作 scope 之外的 stage，系统不靠"它应该知道不该这样"来拦，而是靠 `s ∉ scope → scope_violation` 这条机械规则来拦。这与 P2 的 BFT-lite、P3 的熔断是同一族思路：**把"希望 Agent 守规矩"换成"结构上它就做不到"**。

表1 把 v7.5.0 与 v7.6.0 在权限维度上的差异集中对照。

表 1 owner 模式 vs owner+scope 模式

维度	v7.5.0 (owner)	v7.6.0 (owner+scope)
责任记录	单点责任指针	单点责任指针（保留）
权限记录	隐含=可对全部 todo 下手	显式 scope =todo 的真子集
越界行为	无机械拦截	validate 返回 scope_violation，不产生副作用
任务完成后权限	不自动收回	advance 自动从 scope 移除（收紧）
交接时权限	新 owner 默认继承全部 todo	新 owner 须显式申领并过 validate（重授）
审计	trace 记录事件序列号	额外记录每次 scope 变更与每次越界请求
与既有契约	基线	增量兼容，不推翻 P10 六字段

表1 注：v7.5.0 行为为 P10 已 verified 的契约；v7.6.0 行为为本文设计提案，其防御效果待因子实验测量，[RESULT NEEDED]。

5 与 UDOS 既有篇目的接口

- **对 P10 (TransferBundle)：**scope 是第七字段，不改 goal/context/done/todo/trace 的语义。迁移路径是"加字段、加三条不变量、加一组测试"，不是重写。旧工单没有 scope 字段时，可默认 scope=todo（退回 v7.5.0 行为），保证向后兼容。
- **对 P2 (BFT-lite 停止决策)：**P2 决定"这一棒该不该停"，P17 的 scope 决定"停之前这个 owner 还被允许碰什么"。两者正交：一个是**共识终止**问题，一个是**权限边界**问题。
- **对 P3 (故障治理/熔断)：**scope_violation 是一个比"错误率超阈值"更早的信号——它在越界发生的当下就报警，而不是等滑动窗口累积到阈值。可以作为熔断的前置触发器。
- **对 P11 (内部任务市场)：**P11 的"谁中标、做什么、付多少钱"是经济维度；scope 是这个中标者"被允许动哪些 stage"的能力维度。一个中标者即便在经济上合格，也只能在它的 scope 内执行——**预算够，不等于权限够**。

6 糟粕 / 失效边界 / 可证伪条件

批判性阅读必须落在"这个设计哪里会失灵"上，否则它只是又一段自治的口号。

它的糟粕与局限：

1. **scope 是"结构护栏"，不是"智能判断"**。它能拦住"机械上越权"，但拦不住"在 scope 之内、却把事情做错"。一个 owner 在自己 scope 里做出错误决策，scope 完全感知不到。不要把最小权限误读成"万能安全"。
2. **过度收紧会伤可用性**。如果 scope 切得过细，正常的跨 stage 协作也会被 scope_violation 频繁打断，系统会"因为太谨慎而跑不动"。scope 的粒度本身需要调，而不是越紧越好。
3. **重授依赖新 owner 诚实申领**。new_scope $\subseteq$ todo 是机械可查的，但"new_scope 是否真的刚好够用、不大不小"这一点仍需更高层判断——scope 校验不了 scope 的质量。
4. **ClawArena 的精确数字不能直接搬**。本文借用的是它的方向（权限瓶颈、成本-质量解耦），它的具体阈值（如"无 LLM 超过 50% 权限精度"）未在摘要级逐字命中，且其场景是文本+多模态工作区、不含分布式故障与拜占庭行为——它测的是"**正常情况下好 Leader 能不能管好团队**"，不测"**Leader 被误导时系统崩不崩**"。scope 的故障容错价值，UDOS 必须自己测，不能借 ClawArena 的场景背书。

可证伪条件（设计提案也必须可被杀死）：

- **F1**：如果在因子实验中，owner+scope 相对纯 owner 在越界拒绝率上没有显著改善（Wilson CI 跨 0），那么 scope 的"自动收紧"就是多余复杂度，应回退到 owner 模式。
- **F2**：如果 scope 模式把正常任务的完成率或延迟显著拉低（例如越界误判把正常协作也拦了），且这个代价大于越界收益，那么"最小权限"在工单场景就是负收益设计。
- **F3**：如果发现真正的管理瓶颈根本不在权限授予、而在别处（例如任务分解质量），那么本文把 scope 当成第一抓手就是错位——这正是 EPOB 式"失败分布而非只看成功率"的诊断要回答的问题。

7 预注册实验与诚实边界

scope 的收益目前是**设计提案·待实施**。本文不填任何"降本 X%" "降错 Y%" 的数字。建议的预注册因子实验框架为：

- **因子**：权限模式 {owner 基线, owner+scope} × 链长 {1,2,3,5} × 故障注入 {无, 越权注入, 被误导的诚实 owner}。
- **指标**：scope_violation 触发次数、越界写是否真的被拒（I2 命中率）、正常任务完成率、完成延迟、误隔离率。
- **判据**：在保持正常完成率不显著下降（F2 门槛）的前提下，越界写被拒率是否显著上升（F1 判据）。
- **数字状态**：全部结果位 [RESULT NEEDED: scope 因子实验]，待实验跑完再补，投稿前需 ≥30 seed 并报告 95% CI。

8 结论

v7.5.0 的 TransferBundle 解决了"无人收口"，v7.6.0 的 owner+scope 试图解决"有权限越界"。它的动机来自 ClawArena-Team 的一条方向性提示：管理瓶颈在权限授予，成本与管理质量解耦——所以正确的杠杆是"把权限切小、随流程收紧、交接重授"，而不是"把 Leader 模型换大"。

本文没有声称这已经被实测证明。相反，本文把 scope 写成三条可测试的不变量、一组待跑的因子实验，并把每一处预期收益都标成 [RESULT NEEDED]。结构先验决定上限——owner 这种结构选择决定了系统会在哪类失败上翻车；而把 owner 升级为 owner+scope，就是把"希望 Agent 守规矩"进一步换成"结构上它就越不了界"。这是 P17 对 v7.6.0 的那一点增量。

参考文献

外部对话对象（存在性已于 2026-09-21 联网核验；其精确阈值为论文报告，未独立复核）

1. ClawArena-Team: Benchmarking Subagent Orchestration and Dynamic Workflows in Language-Model Agents. arXiv:2606.31174（论文方向：管理瓶颈集中在权限授予、成本与管理质量近似解耦、纯执行式评分无 LLM 评判；"无 LLM 超 50% 权限精度""成本百倍/总分四倍""SMS 三项乘积公式"等精确阈值未在摘要级逐字命中，按"论文报告·未独立复核"转述）。

经典背景

1. Saltzer, J. H., Schroeder, M. D. (1975). The Protection of Information in Computer Systems. Proceedings of the IEEE（最小权限/访问控制经典；本文只借用其"最小权限"原则，不重新论证）。

UDOS 论文卷（本系列，证据等级随文标注）

1. UDOS v7.5.0 论文卷：P2《多智能体停止决策的 BFT-lite 安全性验证》、P3（故障治理/熔断，v7.5.0 对应篇）、P10《Agent-as-Tool 与 TransferBundle 交接信息损失的可测量化》（六字段契约、26 项测试 verified）、P11《内部任务市场的结算守恒与拜占庭罚没机制设计》、P12《证据分级驱动的可复现 Agent 系统工程》。
 2. UDOS v7.6.0 论文卷：P16《类型化最终性》、P32《规则优先于学习》（scope_violation 属规则谓词，与 P32“规则为底、学习为补”取向一致）。
-

证据纪律与复现说明

- 本文为**设计提案**，无新增 UDOS 实测；scope 三条不变量与因子实验均为待实施设计，全部收益位 [RESULT NEEDED: scope 因子实验]。
- ClawArena-Team 的精确阈值（权限精度百分比、成本/质量倍数、SMS 公式）未在本次摘要级核验中逐字命中，本文不写死任何具体百分比，只引用其方向性结论；引用时须保留“论文报告·未独立复核”口径。
- v7.5.0 既有契约（六字段、26 项测试）回指 P10，证据等级 verified；scope 为其上的增量字段，向后兼容（缺省 scope=todo 即退回 owner 基线）。
- 本文不主张“最小权限=万能安全”；scope 只拦结构上的越界，不拦 scope 之内的错误决策，也需防过度收紧伤可用性（见 §6 糟粕与 F2 判据）。